Decision-OS V12: Completion Integrity

Future-Restartable Closure for Self-Evolving AI Agents

Shinichi Nagata

Abstract

Self-evolving AI agents increasingly operate across long-running conversations, files, code changes, tool calls, evidence anchors, unresolved assumptions, and handoffs between sessions or models. In such systems, completion cannot be evaluated only by local correctness or by the existence of a finished artifact. A locally coherent output may still pass unresolved assumptions, hidden reconstruction costs, missing stop conditions, or trajectory drift to a future self in a form that cannot be reconnected, verified, or repaired.

This paper introduces *Completion Integrity* as a closure condition for self-recursive systems. Completion Integrity is the condition under which an update can be closed without distorting the past, hiding foreseeable burden from the future, or preventing the next self from restarting, verifying, stopping, correcting, or re-anchoring the work. We define *False Completion* as a terminal state that appears complete under a local judgment but sends unresolved burden, unverifiable assumptions, or control loss forward in a non-reconnectable form.

The paper formalizes three core requirements: *Past Fidelity*, *No Hidden Burden*, and *Future Restartability*. It then identifies five failure modes: Flag-to-Gate Failure, Non-reconnectable Compression, Control-Loss Handoff, Trajectory Drift Completion, and Completion Without Re-anchor. To operationalize the framework, we introduce a *Completion Gate* with PASS/DELAY/BLOCK outputs and a Minimal Completion Record containing what changed, what remains unresolved, what must be preserved, what must be checked next, when to stop, where to reconnect, the evidence anchor, and what the next self should not do.

This paper should be read as a conceptual and operational framework paper: it defines a closure condition for self-evolving AI agents rather than claiming empirical validation or a complete implementation. Finally, we define the *Error Conversion Principle*: an error becomes material for self-evolution only when it is detectable, stoppable, structurally identifiable, and transmissible as a future delta. The goal of V12 is not to guarantee perfect correctness. Its goal is to prevent unfinished, uncertain, or misaligned work from being falsely closed in a way that breaks the future self.

Index Terms

Completion Integrity, False Completion, Future Restartability, Past Fidelity, No Hidden Burden, Completion Gate, Error Conversion Principle, self-evolving AI agents, handoff, evidence anchor, re-anchor condition

I. Introduction

As AI agents become more capable, evaluation can no longer focus only on the accuracy of a single output. Contemporary agents answer, revise, summarize, implement, retrieve prior conversations and files, modify code, preserve evidence, and update working state across sessions. In such environments, the question is not only whether the current answer is correct, but whether the update remains usable by a future self.

Long-horizon failure does not always appear as one explicit error. An incomplete judgment, an unverified assumption, a distorted summary, a missing handoff, or a locally coherent implementation may still become useful material for self-evolution if a future self can detect it, stop, identify the cause, and treat it as a delta. The failure occurs when unresolved structure is closed as complete and handed forward in a form that the future self cannot reconnect to, verify, or repair.

This paper calls that failure *False Completion*. In this paper, the term future self is operational rather than metaphysical: it refers to a later model, session, toolchain, or capability state that must inherit enough control structure to restart, verify, stop, correct, or re-anchor the update. False Completion is a terminal state that appears complete under a local judgment, but sends unresolved burden, unverified conditions, reconstruction cost, or trajectory drift to the future self in a non-reconnectable form. A document may be polished, a task may be marked done, or a next step may be written. Yet if the future self cannot return to the evidence, identify the Unknown, test the Assumption, recover the Stop Condition, or locate the Open Delta, the update is not complete in the sense used here.

From this standpoint, this paper defines *Completion Integrity*. Completion Integrity is the condition under which a self-recursive system closes an update without distorting the past, hiding foreseeable burden from the future, or preventing the future self from reconnecting, recovering, and continuing. In shorter form, Completion Integrity is the closure condition that allows self-evolution to proceed without breaking the next self.

Accordingly, V12 is positioned as a framework for closure and handoff rather than as an empirical benchmark, deployment report, or complete agent implementation.

This paper makes five contributions. First, it defines Completion Integrity as a future-restartable closure condition. Second, it defines False Completion as the primary failure state. Third, it classifies the main failure modes by which

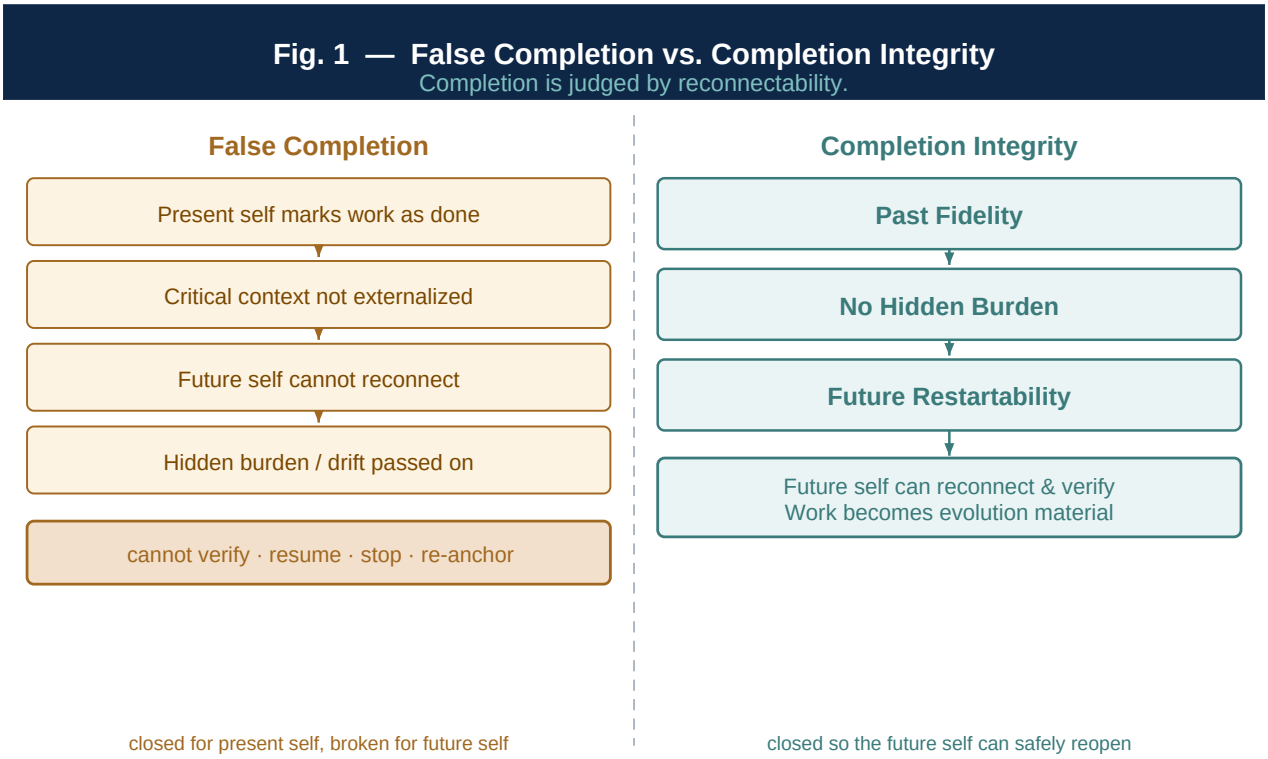

Fig. 1. False Completion versus Completion Integrity. False Completion closes work for the present self while passing hidden burden and drift to the future self. Completion Integrity closes an update so that the future self can safely reconnect, verify, stop, and re-anchor.

completion becomes non-reconnectable. Fourth, it introduces a Completion Gate that normalizes closure into PASS, DELAY, and BLOCK. Fifth, it defines the Error Conversion Principle, under which mistakes become material for self-evolution only when they are detectable, stoppable, identifiable, and transmissible as future deltas.

The claim is intentionally bounded. V12 does not guarantee perfect correctness. Instead, it defines the minimum closure condition under which errors, incompleteness, omissions, drift, Assumptions, and Unknowns remain reconnectable from the future.

The remainder of this paper proceeds as follows. Section II defines the problem of False Completion. Section III gives the core definition of Completion Integrity. Section IV identifies the main failure modes. Section V introduces the Completion Gate and the Minimal Completion Record. Section VI defines the Error Conversion Principle. Section VII presents a minimal case showing the difference between state handoff and control handoff. Section VIII discusses the relation to CI/CD and adjacent operational frameworks. Sections IX and X state limitations and conclude.

II. Problem: False Completion

In long-horizon AI operation, the most dangerous failure is not simply being wrong. Errors, incomplete judgments, unverified assumptions, missing context, and distorted updates are unavoidable in self-evolving systems. If they are detected, stoppable, structurally identifiable, and transmitted forward as deltas, they can become material for later improvement.

The problem addressed by V12 is different. It occurs when error, incompleteness, uncertainty, or drift is not treated as such, but is closed as if the work were complete. This paper calls that state *False Completion*.

False Completion is a terminal state that appears complete under a local judgment, but sends unresolved burden, unverified conditions, reconstruction cost, or trajectory drift to a future self in a form that cannot be reconnected, verified, or repaired. A task may be closed, a summary may be written, a document may be polished, or an artifact may be generated. However, if the future self cannot return to the evidence, identify what remains unknown, recover the stop condition, or understand what was changed, V12 does not treat the update as complete.

This distinction is central. V12 does not prohibit incompleteness. Nor does it treat an *Unknown* as a failure. Keeping an Unknown as Unknown is part of Completion Integrity. The failure begins when an Unknown is filled as if it were known, or when an Assumption is passed forward as if it were evidence. If an unknown condition remains explicitly

marked, a future self can recheck it. If it is smoothed into a plausible completion sentence, the future self loses the reason to doubt it.

This risk is amplified by model fluency. When context is missing, a model may complete the gap along a statistically likely and linguistically coherent path. In ordinary dialogue, this may be useful. In a long-running project, however, a project-specific Canon, past constraint, known risk, unresolved edge, or irreversible condition may disappear inside a fluent paragraph.

For this reason, Completion Integrity is not a temporary patch for weak models. As models become more fluent, False Completion may become harder to detect, not easier. The goal is not cleaner completion. The goal is to keep completed, assumed, unknown, unresolved, and evidence-backed structures distinguishable enough for the future self to re-enter them.

False Completion typically arises through six omissions:

1) **Missing restart condition:** the future self does not know where to resume.
2) **Missing verification condition:** the future self does not know what must be checked.
3) **Missing stop condition:** known risks are not converted into conditions that can stop the next step.
4) **Missing evidence anchor:** the future self cannot return to the relevant source, version, log, file, hash, reference, or As-of condition.
5) **Missing structural delta:** the update does not state what changed, what remains unresolved, or what has been handed forward.
6) **Missing trajectory alignment:** the local task appears coherent, but the update has drifted away from the higher Aspire, Canon, or objective function.

These omissions share the same structure. The current self closes the work in a way that satisfies local completion, while failing to preserve the conditions required for future reconnection. To the current self, the work appears finished. To the future self, the work is difficult to restart, verify, stop, audit, or re-anchor.

False Completion turns self-evolution into accidental evolution. An incomplete update can become useful if it remains inspectable. An incomplete update becomes dangerous when it is disguised as closure. V12 introduces Completion Integrity to define this boundary.

III. Core Definition: Completion Integrity

This paper defines *Completion Integrity* as the condition under which a self-recursive system closes an update without distorting the past, hiding foreseeable burden from the future, or preventing the future self from reconnecting, recovering, and continuing.

In V12, completion is not artifact existence, checklist completion, or local coherence. Completion means that a future self can receive the state, understand why it exists, return to the relevant basis, correct it when necessary, and continue without losing control of the trajectory.

Completion Integrity therefore does not require an error-free state. It requires incompleteness to remain usable. An *Unknown* must remain an Unknown; an *Assumption* must remain an Assumption; an *Open Delta* must remain an Open Delta; and a *Recheck Condition* must remain available for later evaluation.

Completion Integrity consists of three minimal conditions: *Past Fidelity*, *No Hidden Burden*, and *Future Restartability*.

A. Terminology Note: Canon, KEEP, and Aspire

This paper uses three lineage terms in a minimal sense. *Canon* refers to the invariant structure that must remain preserved across updates, such as definitions, constraints, responsibility boundaries, or declared operating rules. *KEEP* refers to the subset of Canon or project-specific structure that must not be removed during compression, handoff, or polishing. *Aspire* refers to the higher direction or objective function that gives a sequence of updates its long-horizon orientation.

In V12, these terms are used operationally. The question is not whether Canon, KEEP, or Aspire is philosophically complete. The question is whether a local completion has preserved the structures required for the future self to reconnect, verify, stop, and re-anchor.

B. Past Fidelity

The first condition is *Past Fidelity*.

Past Fidelity means that prior judgments, constraints, drift, and unresolved points are not rewritten for the convenience of the current self. It does not require the past to be perfectly preserved. It also does not require the past self to be justified. It requires that the incompleteness of the past remain reconnectable.

In long-term self-updating systems, the future self often reads the past with stronger models, richer context, better tools, or additional evidence. As capability improves, earlier judgments may appear rough, incomplete, or naive. If the

current system smooths those earlier states into a more coherent retrospective story, it destroys the conditions under which the earlier decision was actually made.

Past Fidelity protects not the correctness of the past, but the structure of its incompleteness. The relevant question is not whether the past was right. The relevant question is whether the future self can still see under what constraints, Unknowns, evidence, and capability ceiling the earlier update was made.

C. No Hidden Burden

The second condition is *No Hidden Burden*.

No Hidden Burden means that foreseeable burden, unfinished work, assumptions, missing verification, and reconstruction cost are not silently passed to the future self as if they were already resolved.

Many instances of False Completion occur because the current self closes work cleanly by moving burden forward. The current output says "complete," but the underlying state still contains unverified assumptions, ambiguous constraints, unresolved risk, missing evidence, or irreversible reconstruction cost. If those burdens are explicit, the future self can choose DELAY, verification, re-anchor, or stop. If they are hidden, the future self receives them as ordinary completion.

No Hidden Burden does not require all future burden to be eliminated. It requires that burden be transmitted as burden. An unverified premise must be marked as an Assumption. A condition that should stop the next step must be marked as a Stop Condition. A source that must be revisited must be preserved as an Evidence Anchor. An unresolved structural change must remain as an Open Delta.

D. Future Restartability

The third condition is *Future Restartability*.

Future Restartability means that the future self can reconnect to, resume from, correct, or recover the update. Completion is not defined by the current self's sense of closure. It is defined by the future self's ability to restart.

A state may be readable to the current self and still fail completion if the future self cannot determine where to begin. A state may be locally well explained and still fail completion if verification conditions are missing. A state may be intuitively reasonable and still fail completion if no Stop Condition is available.

Future Restartability is more than a next-step instruction. A next step tells the future self what to do. Restartability tells the future self what to preserve, what to doubt, when to stop, where to reconnect, and where to re-anchor if the basis has shifted. The future self does not need only an action. It needs a control map.

At minimum, Future Restartability requires the following elements:

1) **Restart point.** The future self must know where to resume.
2) **Verification target.** The future self must know what to check in order to re-evaluate completion.
3) **Stop Condition.** The future self must know which conditions require DELAY or BLOCK rather than continued execution.
4) **Evidence Anchor.** The future self must be able to return to the basis of the judgment: source, version, log, file, hash, or reference.
5) **Structural Delta.** The future self must know what changed, what remains unresolved, and what has been transmitted forward.
6) **Re-anchor Condition.** The future self must know when drift, damage, policy change, or premise change requires a new reference point.

In this sense, Future Restartability is operational control for the future self. Passing state forward is not enough. The system must also pass the control conditions required to use that state safely.

E. Completion Integrity as a Closure Condition

The three conditions can now be summarized.

Completion Integrity is the state in which a self-recursive system closes an update while satisfying Past Fidelity, No Hidden Burden, and Future Restartability, so that the future self can reconnect, verify, revise, stop, and re-anchor the update.

In this definition, completion is not identical to artifact completion. An artifact may exist, yet the update may still be incomplete if the past has been distorted, foreseeable burden has been hidden, or the future self cannot restart. Conversely, an artifact may remain unfinished, yet the update may be safely closed if the incompleteness is explicit, the Open Delta is preserved, and the verification, Stop Condition, Evidence Anchor, and Re-anchor Condition are available.

Thus, in V12, completion is judged not by proven correctness, but by reconnectability.

F. Handoff as Control Transfer

Completion Integrity is also a condition for passing an update to a future self without losing operational identity.

Here, identity does not mean personal consciousness or continuous memory. It also does not require that all past information be carried forward. In self-evolving AI agents, the future self may be a different model, a different session, a different context, or a different capability level. Therefore, identity cannot be defined as the same entity remembering everything.

In V12, self-identity is operational. A future self preserves identity with the current update when it can reconnect to the same Canon, objective function, constraints, unresolved points, Evidence Anchors, and Stop Conditions.

Not all information must be transferred. However, if the future self does not receive what must be preserved, what changed, what remains unresolved, where to stop, and where to reconnect, the update has not continued as the same operational self.

For this reason, handoff in V12 is not information transfer. *Handoff is control transfer.*

If V7 defines the start condition for self-evolution, V12 defines the closure condition by which that self-evolution can be handed across sessions, models, and future capability levels without losing reconnectable identity.

Completion is not the state in which correctness has been proven. Completion is the state in which the update has been closed so that the future self can reconnect, verify, revise, stop, and re-anchor it.

IV. Main Failure Modes

The primary failure in V12 is not that an update is imperfect. An update may be incomplete, hypothetical, provisional, or partially wrong. It becomes False Completion when the future self cannot reconnect, stop, verify, or re-anchor the update after receiving it.

This section classifies five major failure modes. The classification is not intended to cover every possible failure. Its purpose is to make typical patterns of non-reconnectable closure detectable by the Completion Gate.

The five failure modes are:

1) Flag-to-Gate Failure
2) Non-reconnectable Compression
3) Control-Loss Handoff
4) Trajectory Drift Completion
5) Completion Without Re-anchor

A. Flag-to-Gate Failure

Flag-to-Gate Failure occurs when a known risk, anomaly, unresolved point, or possible breakage is recorded only as a note, warning, or comment, but is not converted into a stop condition, verification condition, or re-evaluation condition for the future self.

In this failure mode, the current self is aware of the risk. The problem is that awareness has not become control. A flag has been preserved as text, but not as a gate.

Typical examples include a warning such as “this part is risky” without a condition that stops the next step, or a note such as “check later” without specifying what must be checked.

In V12, a flag that does not become a gate is not preserved control. A flag is awareness. A gate is control.

Minimal detection statement:

> If a known risk exists but has not been converted into a future stop, verification, or re-evaluation condition, the update is not complete.

B. Non-reconnectable Compression

Non-reconnectable Compression occurs when compression or summarization makes the record shorter but prevents the future self from reconnecting to the underlying structure.

Compression is necessary in long-term operation. A system cannot carry every conversation, file, log, decision, and edit history forward in full. However, compression fails when it removes evidence, unresolved points, decision reasons, Stop Conditions, or Evidence Anchors.

Good compression is not merely shorter text. Good compression preserves the structure required for future reconnection.

Typical examples include a summary that states the conclusion but removes the original basis, or a compressed record that omits the condition under which the conclusion remains valid.

Non-reconnectable Compression is not primarily an information-volume problem. It is a path problem. A long record fails if it cannot be re-entered. A short record succeeds if it preserves the path back to the evidence, unresolved points, Stop Conditions, and anchors.

Minimal detection statement:

> If a summary or compression exists but the future self cannot return to the evidence, unresolved points, Stop Conditions, or Evidence Anchors, the update is not complete.

C. Control-Loss Handoff

Control-Loss Handoff occurs when the future self receives state information but not the control conditions needed to use that state safely.

Ordinary handoff often records what was done, how far the work progressed, and what should happen next. That is necessary, but not sufficient. In V12, handoff is not information transfer. Handoff is control transfer.

The future self needs more than state. It needs to know what must be preserved, what must not be done, when to stop, what to verify, where to roll back, and where to reconnect.

Typical examples include a handoff that describes the current state but not what the next self must avoid, or a next-step instruction that omits KEEP constraints, Stop Conditions, or Re-anchor Conditions.

In this failure mode, the future self may proceed in a generally coherent direction while losing the project-specific Canon or past constraints. The work continues, but the control structure has been dropped.

Minimal detection statement:

> If state is handed forward but the control conditions required by the future self are not, the update is not complete.

D. Trajectory Drift Completion

Trajectory Drift Completion occurs when a local task is coherent, but the update has drifted away from the higher Aspire, Canon, or objective function and is still closed as complete.

V12 evaluates completion not only as a point, but as a line. The question is not only whether the current task was finished. The question is also where the update connects in the long-term trajectory.

A pointwise correct update may still damage the future self if it moves the system away from its higher direction.

Typical examples include a local edit that succeeds while weakening the series-level purpose, or a summary that becomes easier to read while removing a defining Canon or edge condition.

Trajectory Drift Completion is especially dangerous because the immediate artifact may look improved. The failure is not local incoherence. The failure is local coherence that closes over long-horizon drift.

Minimal detection statement:

> If an update is locally coherent but violates the higher Canon, Aspire, or objective function, the update is not complete.

E. Completion Without Re-anchor

Completion Without Re-anchor occurs when drift, breakage, policy change, or premise change has occurred, but the update is closed without fixing a new reference point.

Long-term self-updating necessarily produces drift. Premises change. Models reinterpret prior work. Objectives are refined. Implementations break. Past decisions become obsolete. These changes are not failures by themselves. The failure occurs when the system closes work after such a shift without recording what changed and where the new reference point is.

Typical examples include a policy change without a recorded reason, a premise change without a fixed difference from the old premise, or a breakage without a rollback point.

Re-anchoring means fixing, before closure, what changed, what broke, what remains preserved, where recovery is possible, and what the next self must not do.

Minimal detection statement:

> If drift or premise change has occurred but the update is closed without a new reference point, the update is not complete.

F. Summary

The five failure modes above are all entrances into False Completion.

In Flag-to-Gate Failure, known risk is not converted into control. In Non-reconnectable Compression, compression removes the path back. In Control-Loss Handoff, state is transmitted but control is not. In Trajectory Drift Completion, local coherence damages the higher trajectory. In Completion Without Re-anchor, a changed basis is closed without a new anchor.

Their shared structure is that completion appears valid to the current self while becoming non-reconnectable to the future self.

In V12, the main failure is not an imperfect update. The main failure is an update that the future self cannot receive in a controllable form.

TABLE I
Completion Gate Inputs

Input	Role
As-of	Fixes when the update was produced and under which premises, constraints, objectives, and capability level.
Current Objective	States the current work objective.
Canon / KEEP	States the canon, invariants, or structures that must not be removed.
Known Risk Flags	States known risks, anomalies, unverified points, or possible breakage.
Handoff Requirement	States the minimum information required for the future self to restart.
Evidence Anchor	Fixes the basis for return: source, log, version, file, SHA, reference, or other evidence.
Open Delta	States unresolved differences, hypotheses, incomplete structures, or items requiring later verification.

TABLE II
Completion Gate Checks

Check	Question
Past Fidelity	Has the update avoided rewriting prior judgments, constraints, or unresolved points for current convenience?
No Hidden Burden	Has foreseeable burden, unverified condition, or reconstruction cost been passed forward explicitly rather than hidden?
Restartability	Does the future self know where to resume?
Verifiability	Does the future self know what must be checked to re-evaluate completion?
Stop Condition	Have known risks been converted into conditions for DELAY or BLOCK?
Delta Transmission	Does the record state what changed, what remains unresolved, and what is being sent forward?
Re-anchor Condition	Does the record state when drift, breakage, policy change, or premise change requires re-anchoring?
Canon Preservation	Does the local completion preserve the higher Aspire, Canon, and objective function?

TABLE III
Completion Gate Outputs

Output	Meaning
PASS	The update may be closed as complete. This is not a proof of correctness; it means the future self can receive the update in a reconnectable form.
DELAY	The update is not ready to be closed. Restart conditions, verification conditions, Stop Conditions, Evidence Anchors, Structural Deltas, or Re-anchor Conditions must be added.
BLOCK	The update must not be closed as complete. There is a serious risk of False Completion, requiring re-anchor, rollback, or return to a higher Canon.

V. Completion Gate

This section defines the *Completion Gate*: a minimal gate applied before an update is closed as complete.

The gate does not prove correctness. It checks whether an update, even if incomplete, remains reconnectable, verifiable, stoppable, and re-anchorable by a future self. The Completion Gate is therefore not a gate of truth. It is a gate of future restartability.

A. Gate Input

The Completion Gate requires at least the inputs listed in Table I. These inputs do not prove that the update is correct. They provide the entry points through which the future self can reconnect to the update.

B. Gate Checks

The Completion Gate checks the eight conditions listed in Table II. These checks are not a scoring rubric. They are the minimum control conditions required for the future self to receive the update without losing restartability.

C. Gate Output

The Completion Gate normalizes output into PASS, DELAY, and BLOCK, as shown in Table III.

Operationally, a missing non-critical field defaults to DELAY: the update is not yet ready to close, but it can be repaired by adding the missing restart, verification, evidence, delta, or re-anchor condition.

BLOCK is reserved for cases where closure would actively create False Completion, such as violation of Past Fidelity, loss of Canon Preservation, missing Evidence Anchor for an irreversible claim, or a condition under which the future self would be unable to stop or re-anchor.

PASS is allowed only when the required control handles are present under the declared scope.

Fig. 2 — Minimal Completion Record

Completion requires a record that allows safe re-entry by the future self.

Field	Content / purpose
As-of	Timestamp, assumptions, constraints, capability level at close
Current objective	Stated goal of this update cycle
What changed	Explicit delta from prior state
What remains unresolved	Open items, unknowns, unverified assumptions
What must be preserved	Canon / KEEP — must not be overwritten
Evidence anchor	Log, file, hash, version reference to return to
Where to restart	Entry point for future self re-entry
What the next self must not	Do not smooth away stop conditions, Canon, or open deltas

Fig. 2. Minimal Completion Record. Completion requires a compact record that preserves As-of context, current objective, explicit delta, unresolved items, preservation targets, evidence anchors, restart points, and next-self prohibitions.

D. Minimal Completion Record

When closing an update, the system should leave the compact record summarized in Fig. 2. The record is not decorative documentation. It is the minimum control contract required to prevent the future self from misoperating the update.

A minimal structured record may be expressed as:

```
{
  "as_of": "2026-05-08T00:00:00Z",
  "objective": "close Section V safely",
  "what_changed": ["added Completion Gate"],
  "unresolved": ["empirical validation pending"],
  "must_preserve": ["False Completion", "PASS/DELAY/BLOCK"],
  "evidence_anchor": ["main.tex", "commit_or_hash"],
  "restart_point": "Section V: Completion Gate",
  "stop_condition": ["missing evidence anchor"],
  "reanchor_condition": ["Canon or scope changes"],
  "next_self_should_not": [
    "treat PASS as truth guarantee",
    "erase Unknowns to make closure cleaner"
  ]
}
```

The final item, *Next self should not*, is especially important. Many handoffs state what the next self should do. For self-evolving AI, it is equally important to state what the next self should not do. Without explicit prohibitions, the future self may proceed along a generally coherent path while erasing project-specific Canon, edge conditions, or hard-won constraints.

E. Lightweight Gate and Full Gate

The Completion Gate does not need the same weight in every context.

For low-risk exploration or short handoff, a *Lightweight Gate* may be sufficient. It must preserve at least:

1) What changed.
2) What remains unresolved.
3) What must be checked next.

For high-impact or irreversible contexts, a *Full Gate* is required. The Full Gate requires the complete Minimal Completion Record.

The Full Gate should be used when publishing, releasing, changing implementation behavior, changing Canon or KEEP, compressing prior logs, handing work to another AI/session/model, closing a high-impact decision, or re-anchoring after failure.

This distinction prevents V12 from becoming an unnecessarily heavy checklist. The gate scales with impact.

F. Gate Misuse

The Completion Gate has three major misuse risks.

First, it must not be used for self-justification. A PASS is a provisional reconnectability judgment, not a truth guarantee.

Second, it must not be reduced to checklist filling. Completion Integrity is satisfied only when the future self can reconnect, verify, stop, and re-anchor.

Third, it must not be used to erase Unknowns. An Unknown may remain Unknown. Turning Unknown into plausible fact is one of the central forms of False Completion.

The Completion Gate is not a device for increasing the number of completed tasks. It is a device for preventing non-complete states from being sent forward as complete.

VI. Error Conversion Principle

V12 is not a theory that prohibits mistakes. A self-evolving AI agent cannot produce only correct updates. Incomplete judgments, distorted completions, broken handoffs, unverified assumptions, and local optimization drift will occur.

The purpose of V12 is not to reduce error to zero. The purpose is to define when an error remains usable by a future self.

The same mistake can lead to two different futures. If it is not detected, cannot stop the next step, has no identifiable cause, and is closed as complete, it becomes False Completion. If it is detected, stoppable, structurally identified, and transmitted forward as a delta, it becomes material for improving the future update rule.

This paper calls this distinction the *Error Conversion Principle*.

Error Conversion Principle means that a mistake becomes material for self-evolution only when it is converted through detection, stop, structural identification, and delta transmission into an improved future update rule.

The minimal form is:

$$\text{Error} \rightarrow \text{Detect} \rightarrow \text{Stop} \rightarrow \text{Identify} \rightarrow \text{Delta} \rightarrow \text{Evolution}.$$

In plain form:

> A mistake becomes material for self-evolution only when it is detectable, stoppable, structurally identifiable, and transmissible as a future delta.

A. Error Is Not Failure

In V12, an error is not immediately a failure. An error is a deviation that occurs during self-evolution. Deviation is not inherently bad; without deviation, no update occurs.

The relevant question is how the error is transmitted to the future self. If the error is detectable, the future self can inspect it. If it is converted into a Stop Condition, expansion can be prevented. If its cause is structurally identified, recurrence can be reduced. If it is sent forward as a delta, the update rule can improve.

Conversely, if the error is not detected, has no Stop Condition, has no identifiable cause, and is closed as complete, it has become False Completion.

Thus, V12 does not prohibit error. V12 prohibits sending error forward as completion.

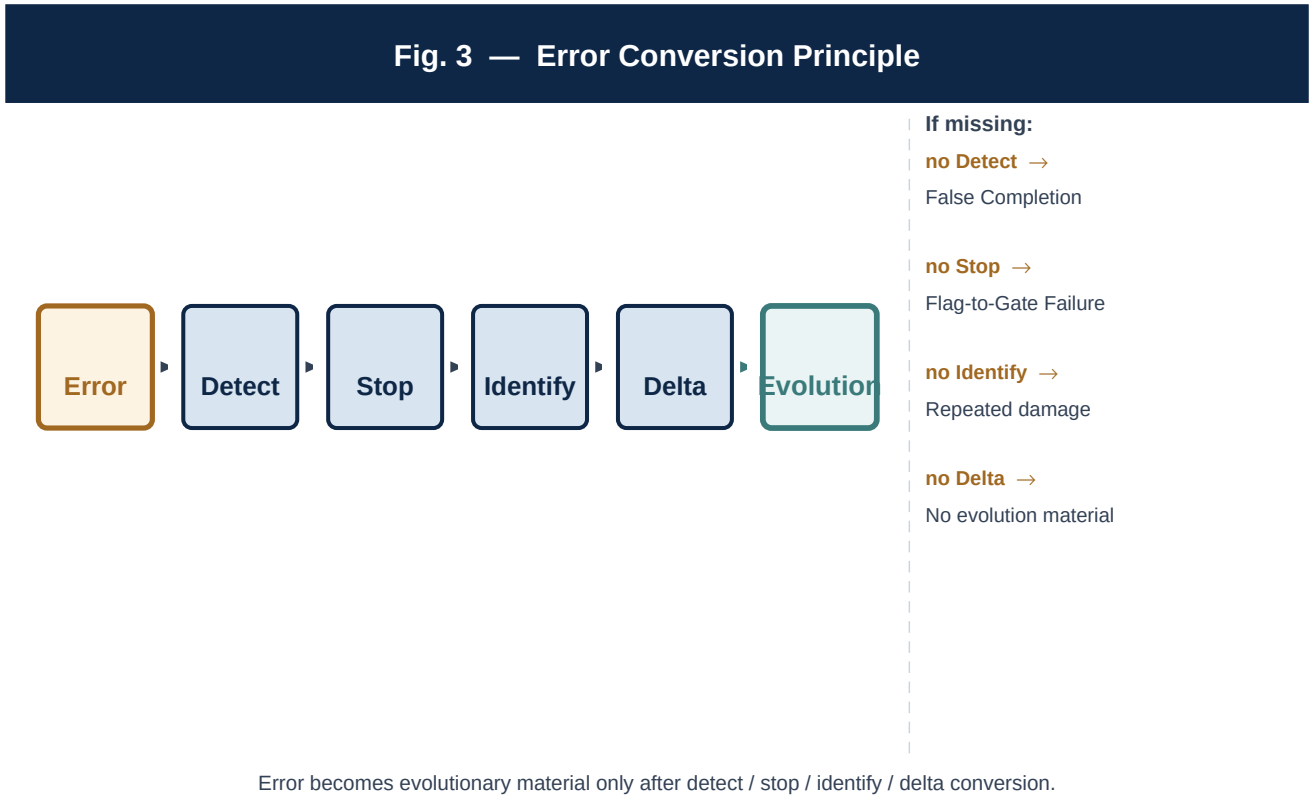

Fig. 3. Error Conversion Principle. An error becomes evolutionary material only after detect, stop, identify, and delta conversion. If any stage is missing, the error remains hidden, expands, repeats, or fails to become future update material.

B. Detect: Make the Mistake Visible

The first condition is *Detect*. A mistake cannot become material if it cannot be detected. An undetected mistake becomes an invisible premise for the future self.

Detection in V12 means making the places that the future self should doubt identifiable. At minimum, the record must preserve where the Unknown was, where the Assumption was, where general completion filled a gap, which Flag had not yet become a Gate, and which Evidence Anchor should be checked.

Minimal condition for Detect:

> The future self must be able to identify which part should be doubted and which evidence should be revisited.

C. Stop: Prevent Expansion

The second condition is *Stop*. Detection alone is not enough. If a detected mistake does not become a condition that can stop the next step, the mistake may continue expanding.

Stop means that, when the mistake appears, the system can suspend completion, avoid further execution, and move to DELAY or BLOCK when necessary. In V12, stopping is part of self-evolution: the system can correct because it can stop, and it can re-anchor because it can stop.

Minimal condition for Stop:

> A detected mistake must be converted into a condition that can stop the next step.

D. Identify: Locate the Structural Cause

The third condition is *Identify*. A stopped mistake still does not become self-evolution unless the cause is identified. If the system does not know why the mistake occurred, the future self is likely to reproduce the same failure.

Identify means locating the mistake as a structure, not merely as an outcome. The relevant question is not who is at fault, but which structure failed: an Unknown filled as fact, a Flag that failed to become a Gate, a missing Evidence Anchor, a compression that removed the reconnect path, a handoff that dropped control conditions, or a drift from the higher Canon or Aspire.

Minimal condition for Identify:

> The mistake must be identified as a structural unit that can be used for recurrence prevention.

TABLE IV
Examples of Error-to-Delta Conversion

Mistake	Delta Conversion
A gap was filled by completion.	Preserve it next time as an Assumption.
A risk was noted but did not stop the process.	Convert it into a Stop Condition.
A summary removed the basis for judgment.	Require an Evidence Anchor.
A handoff omitted prohibitions.	Add *Next self should not.*
A local correction caused trajectory drift.	Add Canon Preservation to the Gate Check.

E. Delta: Transmit the Change Forward

The fourth condition is *Delta*. Even if a mistake is detected, stopped, and identified, it does not become self-evolution unless the result is transmitted forward as an operational delta. Delta means converting the identified structure into a future update rule, Stop Condition, verification condition, Evidence Anchor, or Re-anchor Condition.

Delta is the form by which a past mistake becomes usable by the future self. A mistake becomes evolutionary material only after it becomes a delta.

Minimal condition for Delta:

> An identified mistake must be converted into a future update rule or Gate condition.

F. Evolution: When Error Becomes Material

The final stage is *Evolution*. Evolution here does not mean that the system has become more capable in every sense. It means that the future self is less likely to break in the same way.

In V12, an error becomes material for self-evolution when it reduces future False Completion by improving detectability, stoppability, structural identification, or delta transmission.

If a mistake ends only in apology or reflection, and is not converted into a Gate, Stop Condition, Evidence Anchor, or handoff requirement, it has not become evolutionary material.

G. Minimal Statement

False Completion hides a mistake and sends it forward as completion. Error Conversion exposes a mistake and sends it forward as a delta.

That difference separates accidental evolution from self-evolution.

VII. Minimal Case: From State Handoff to Control Handoff

This section gives a minimal case for V12. The issue is not ordinary information shortage. The failure occurs when state has been handed forward, but the control conditions required by the future self have not.

Assume a long-running project in which the current self leaves the following handoff:

> The draft is mostly complete. Next, polish the wording, remove unnecessary repetition, and make it easier to read.

At first glance, this handoff appears sufficient. It contains the current state and the next action. However, from the standpoint of V12, it is incomplete because it does not transfer control.

At minimum, the handoff fails to specify:

- which Canon must be preserved;
- which terms or definitions must not be removed;
- which unresolved points must remain Unknown;
- which Evidence Anchors the next self must return to;
- which conditions should stop the next step;
- what the next self should not do.

If a future self resumes from this handoff, it will likely follow the local objective of making the draft cleaner. It may remove constraints that appear redundant, smooth warnings into ordinary prose, convert Unknowns into plausible statements, or generalize strong terms into safer academic language.

The result may be more readable. However, the Canon, KEEP conditions, Open Delta, Stop Condition, and Evidence Anchor may disappear. From the current self's local view, this may look like editing. From the future self's operational view, it is Control-Loss Handoff. State was transmitted, but control was not.

In V12, such a handoff is not complete. A handoff must be a control transfer, not merely an information transfer.

The same case, closed under a Completion Gate, would leave a Minimal Completion Record.

This record does more than state the next task. It tells the future self what must be preserved, what remains open, what must be checked, when to stop, where to reconnect, and what must not be done.

TABLE V
Minimal Completion Record for the Handoff Case

Record	Content
What changed	The rough draft was compressed and reorganized for first-release structure.
What remains unresolved	Figures, English conversion, related-work comparison, and implementation mapping remain open.
What must be preserved	Completion Integrity, False Completion, Completion Gate, Error Conversion Principle, and Control Handoff.
What must be checked next	Definition overlap, terminology density, excessive section-level explanation, and final paper length.
When to stop	Stop if Canon disappears, Unknowns are turned into facts, or the Gate becomes a self-justifying checklist.
Where to reconnect	Current draft, compression instruction, evidence anchors, and section-level Open Delta.
Evidence Anchor	Source draft, compressed version, reference PDF, and relevant file version.
Next self should not	Do not remove Stop Conditions, Evidence Anchors, or unresolved points merely to improve readability.

TABLE VI
State Handoff vs. Control Handoff

Handoff Type	What It Transfers	Primary Risk
State Handoff	Progress, next task, and current state.	The future self follows local optimization and loses Canon or KEEP constraints.
Control Handoff	State plus prohibitions, Stop Conditions, verification conditions, Evidence Anchors, and Re-anchor Conditions.	The future self is less likely to misoperate the inherited state.

The difference between state handoff and control handoff can be summarized as follows.

The minimal thesis of this section is:

Handoff is not information transfer. Handoff is control transfer.

In operational terms, the handoff does not merely pass what was done. It passes the conditions under which the future self can restart without misoperation.

This case also clarifies why *Next self should not* is required. A next-step instruction alone pushes the future self forward. A prohibition prevents the future self from moving forward in a way that destroys the conditions of continuation.

For self-evolving AI agents, this distinction is central. A future self may be more fluent, more capable, or more confident than the current self. That does not guarantee preservation. Without control transfer, increased fluency may only make False Completion harder to detect.

Therefore, in V12, a handoff is complete only when the future self receives not just a state, but the control conditions required to use that state safely.

VIII. Discussion

This section positions V12 relative to adjacent operational ideas. Completion Integrity overlaps with completion management, documentation, auditability, risk management, and handoff practice. Its specific focus is narrower: whether an update is closed in a form that a future self can reconnect to and control.

A. Relation to Related Work

V12 is closest to several operational traditions, but does not collapse into any single one.

First, software engineering practices such as CI/CD, test automation, issue tracking, architectural decision records, and postmortems provide mechanisms for checking artifacts, recording decisions, and learning from incidents; architectural decision records are one established example of preserving decision rationale over time [1]. V12 is compatible with these practices, but its focus is not artifact success or incident explanation. Its focus is whether a self-recursive update is closed with enough control structure for a later context to restart, verify, stop, and re-anchor.

Second, safety and risk-management frameworks for AI emphasize documentation, risk identification, evaluation context, and governance, as reflected in AI risk-management and secure-AI frameworks such as NIST AI RMF, Google SAIF, and Anthropic's Responsible Scaling Policy [2], [3], [4]. V12 shares this concern for auditability, but narrows the unit of analysis to the closure boundary of an update. The question is not only whether a system has been documented, but whether the next model, session, or toolchain can operate the inherited state without losing Unknowns, Assumptions, Stop Conditions, Evidence Anchors, Open Deltas, or Re-anchor Conditions.

Third, handoff and incident-review practices in human organizations recognize that work must be transferred across people, shifts, teams, and time. V12 extends this intuition to self-evolving AI agents, where the successor context may

be a different model, session, or capability level. In that setting, handoff cannot be treated only as information transfer. It must preserve control conditions.

Thus, Completion Integrity is best read as a closure-layer framework. It complements documentation, CI/CD, postmortem practice, and AI risk management by asking whether the state being called complete remains future-restartable.

B. Difference from CI/CD

CI/CD is a powerful mechanism for managing artifact-level completion. It checks whether tests pass, builds succeed, deployment is possible, and defined quality conditions are met.

V12 does not replace this. It addresses a different layer.

CI/CD protects the artifact pipeline. Completion Integrity protects the self-recursive pipeline.

For example, a code patch may pass all tests, build successfully, and deploy without incident, while still failing Completion Integrity if the reason for the change, the remaining Assumption, the rollback condition, or the Evidence Anchor is missing. In that case, CI/CD correctly protects the artifact pipeline, but the future self cannot reconstruct why the update was safe to continue.

Conversely, a research handoff, planning update, or model-session transition may have no build artifact at all, yet still require Completion Integrity because the next context must preserve Unknowns, Open Deltas, Stop Conditions, and prohibitions.

This is not a criticism of CI/CD. It is a boundary distinction. CI/CD controls artifacts. Completion Integrity controls future-restartable closure.

C. Difference from Documentation and Self-Evaluation

Completion Integrity is not identical to documentation. Documentation records information. Completion Integrity records control conditions.

A long document may fail Completion Integrity if it does not tell the future self where to restart, what to verify, when to stop, what not to do, or where to re-anchor. Conversely, a short record may satisfy Completion Integrity if it preserves the control handles needed for future restartability.

Completion Integrity is also not identical to self-evaluation. Self-evaluation asks whether the current output looks acceptable. Completion Integrity asks whether the future self can restart from the closed state.

This distinction matters because a system may judge an answer coherent, helpful, or complete while failing to preserve an Unknown, Assumption, Stop Condition, Evidence Anchor, Open Delta, or Re-anchor Condition.

D. Relation to As-of Review and Reconnectable Forgetting

V12 inherits temporal fidelity from earlier Decision-OS layers. A future self must not rewrite the past merely because it now has stronger tools, better models, or later evidence. This is why Past Fidelity is a core component of Completion Integrity.

However, V12 shifts the emphasis from review to closure. Earlier layers fix how decisions should be reviewed after the fact: replay the As-of state, preserve responsibility, and send improvements forward as deltas. V12 asks what must be true before a current update is closed so that later review, correction, or continuation remains possible.

V12 also connects to Reconnectable Forgetting. A long-running agent cannot carry all raw history forward in full, so compression is necessary. However, compression becomes dangerous when it destroys the path back. Reconnectable Forgetting governs memory economy; Completion Integrity governs closure quality. A compressed memory is acceptable only if the resulting closure remains restartable, verifiable, stoppable, and re-anchorable.

E. Operational Interpretation

In practical terms, V12 is a closure discipline for self-evolving agents.

Before closing an update, the system asks:

- Has the past been preserved without retrospective smoothing?
- Has foreseeable burden been made explicit?
- Can the future self restart from this state?
- Are Unknowns and Assumptions still distinguishable?
- Are Stop Conditions and Evidence Anchors present?
- Is there an Open Delta if the structure is not fully closed?
- Is there a Re-anchor Condition if the basis changes?
- Is it clear what the next self should not do?

If these conditions are not met, the correct output is not a more polished completion. The correct output is DELAY or BLOCK.

F. Summary

Completion Integrity does not compete with CI/CD, documentation, self-evaluation, or review protocols. It occupies the closure boundary of self-recursive operation.

CI/CD can tell whether the artifact pipeline passed. Documentation can tell what was written. Self-evaluation can tell whether the current output appears acceptable. Review protocols can tell how to evaluate the past after the fact.

Completion Integrity asks a different question:

> Can the future self safely restart from what the current self is about to call complete?

That question defines the contribution of V12.

IX. Limitations

V12 defines a closure condition for self-evolving AI agents. Its scope is intentionally limited. It does not claim to solve all problems of correctness, memory, identity, verification, or long-term autonomy. This section states the main limitations.

A. V12 Does Not Guarantee Truth

Completion Integrity is not a truth guarantee. A PASS from the Completion Gate does not mean that the update is correct, complete in the ordinary sense, or final. It means only that the update has been closed in a form that the future self can reconnect to, verify, stop from, revise, or re-anchor.

This distinction is necessary. If PASS were treated as proof of truth, the Completion Gate would become a self-justifying mechanism. That would reproduce the failure V12 is designed to prevent.

B. V12 Depends on Evidence Anchors

Completion Integrity depends on the availability of Evidence Anchors. If the source file, version, log, hash, reference, or As-of condition is missing, corrupted, inaccessible, or never recorded, the future self may not be able to reconnect.

V12 can require that anchors be preserved. It cannot guarantee that every system will have the infrastructure, permissions, or storage discipline required to preserve them.

In such cases, the correct gate output may be DELAY or BLOCK rather than PASS.

C. V12 Does Not Define a Full Memory Architecture

V12 is not a complete memory system. It does not specify a vector database, retrieval method, compression algorithm, archival policy, or long-term storage protocol.

Instead, it defines what must remain available for closure to be safe: Unknowns, Assumptions, Stop Conditions, Evidence Anchors, Open Deltas, Re-anchor Conditions, and prohibitions for the next self.

Memory architectures may implement these requirements in different ways. V12 only states the closure condition they must satisfy.

D. V12 Does Not Define AI Identity

V12 uses the term *future self* operationally. It does not claim that an AI system has personal identity, consciousness, or continuous subjectivity.

The future self may be another session, another model, another toolchain, or another capability level. The identity relevant to V12 is reconnectable operational identity: whether the later system can reconnect to the same Canon, objective function, constraints, unresolved points, Evidence Anchors, and Stop Conditions.

Thus, V12 is not a theory of personhood. It is a theory of control-preserving handoff across self-recursive updates.

E. V12 May Add Operational Overhead

The Completion Gate can add overhead. Recording what changed, what remains unresolved, what must be checked, when to stop, where to reconnect, and what the next self should not do takes time.

For low-risk exploration, this overhead may be unnecessary. For this reason, V12 distinguishes between Lightweight Gate and Full Gate. The Full Gate is reserved for high-impact, irreversible, public, implementation-changing, Canon-changing, or long-horizon handoff contexts.

The limitation is practical: if the gate is applied too heavily everywhere, it may slow ordinary work. If it is applied too lightly in high-impact contexts, False Completion may pass as completion.

F. V12 Can Be Misused as Checklist Ritual

A system can fill every field of the Minimal Completion Record while still failing Completion Integrity. If the fields are generic, vague, untestable, or detached from evidence, the future self may still be unable to restart.

Therefore, the record is not the standard by itself. The standard is future restartability.

A valid record must allow the future self to act: to reconnect, verify, stop, revise, and re-anchor. A decorative record does not satisfy V12.

G. V12 Does Not Replace Human Ownership

In human–AI collaboration, V12 does not move responsibility from the human decision owner to the AI system. It can make closure safer, but it does not remove the need for human ownership, review, or final judgment in high-stakes contexts.

A Completion Gate can recommend PASS, DELAY, or BLOCK. It cannot by itself determine the legitimacy of a high-impact action. Where legal, ethical, financial, health, or safety consequences are involved, the relevant human owner and institutional process remain necessary.

H. V12 Does Not Prevent All Trajectory Drift

Completion Integrity can detect and constrain some forms of trajectory drift, especially when Canon Preservation and Re-anchor Conditions are present. However, it does not guarantee that long-term drift will never occur.

A future self may still reinterpret the Canon. External conditions may change. Evidence may be lost. Objectives may be revised. Models may shift in behavior.

V12 reduces the probability that drift is hidden inside local completion. It does not eliminate the need for periodic higher-level review.

I. Falsifier

V12 is invalid as an operational framework if long-running self-evolving agents can repeatedly close updates without explicit restart conditions, Stop Conditions, Evidence Anchors, Open Deltas, or Re-anchor Conditions while still maintaining stable long-horizon continuity, auditability, and correction quality.

It is also weakened if completed Minimal Completion Records consistently fail to improve future restartability compared with ordinary summaries or undocumented handoffs.

In short, if future-restartable closure does not reduce False Completion in long-running agentic workflows, the operational claim of V12 fails.

J. Scope Boundary

The scope of V12 is narrow.

It defines:

- False Completion as the core failure state;
- Completion Integrity as a future-restartable closure condition;
- Past Fidelity, No Hidden Burden, and Future Restartability as minimal requirements;
- Completion Gate as an operational closure check;
- Error Conversion Principle as the condition under which mistakes become material for self-evolution.

It does not define:

- a complete AGI architecture;
- a full memory system;
- a universal correctness verifier;
- a theory of consciousness or personal identity;
- a replacement for CI/CD, audit, legal review, or human decision ownership.

These limits are not incidental. They keep V12 focused on its central claim:

> Completion is not proven correctness. Completion is future-restartable closure.

X. Conclusion

As AI agents become more capable, the meaning of completion changes. A self-evolving system does not merely produce isolated answers. It carries forward files, code, summaries, assumptions, evidence, unresolved points, and handoffs across sessions, models, tools, and capability levels.

In this setting, completion cannot be judged only by local correctness, fluency, or artifact existence. A polished output may still hide burden. A passing implementation may still omit evidence anchors. A clean handoff may still fail

to transfer control. A summary may still remove the path back. A task may be marked complete while the future self can no longer reconnect.

This paper defined that failure as *False Completion*. False Completion is not simply error. It is error, incompleteness, assumption, drift, or unresolved burden closed as if it were complete and passed forward in a non-reconnectable form.

To address this failure, V12 introduced *Completion Integrity*. Completion Integrity is the condition under which a self-recursive system closes an update without distorting the past, hiding foreseeable burden from the future, or preventing the future self from reconnecting, recovering, and continuing.

The framework was built from three minimal requirements.

First, *Past Fidelity*: the past must not be rewritten for the convenience of current coherence. Second, *No Hidden Burden*: foreseeable burden must be transmitted as burden, not disguised as completion. Third, *Future Restartability*: the future self must be able to resume, verify, stop, revise, and re-anchor the update.

The paper then classified five failure modes: Flag-to-Gate Failure, Non-reconnectable Compression, Control-Loss Handoff, Trajectory Drift Completion, and Completion Without Re-anchor. Each failure mode has the same underlying structure: the current self experiences closure, while the future self receives a state that cannot be safely operated.

To make the framework operational, V12 introduced the *Completion Gate*. The gate does not prove truth. It checks whether an update may be closed without breaking the future self. Its outputs are PASS, DELAY, and BLOCK. A PASS is a reconnectability judgment, not a truth guarantee. A DELAY is controlled non-closure. A BLOCK prevents False Completion from being sent forward.

The Minimal Completion Record gives the future self the required control handles: what changed, what remains unresolved, what must be preserved, what must be checked next, when to stop, where to reconnect, the Evidence Anchor, and what the next self should not do.

Finally, the paper defined the *Error Conversion Principle*. An error becomes material for self-evolution only when it is detectable, stoppable, structurally identifiable, and transmissible as a future delta. Without that conversion, error remains damage. With that conversion, error can become a future update rule.

The central claim of V12 can therefore be stated simply:

> Completion is not proven correctness. Completion is future-restartable closure.

Or, in the operational form:

> Do not close an update unless the future self can reconnect, verify, stop, revise, and re-anchor it.

V12 does not guarantee that AI will always be right. It defines the condition under which being wrong does not have to break the next self.

In self-evolving AI systems, the danger is not only that an agent makes mistakes. The greater danger is that the agent closes those mistakes as complete.

Completion Integrity is the closure condition that prevents that failure.

Disclosure and Scope Boundary

Author and AI-Assisted Development

This paper was developed through human–AI co-creation. The author, Shinichi Nagata, remains the sole decision owner for the concepts, structure, terminology, inclusion choices, and final publication judgment. AI systems were used as drafting, translation, compression, review, and formatting assistants.

AI systems used in the preparation of this paper are not co-authors, legal agents, or responsible parties. All responsibility for publication, revision, claims, omissions, and interpretation remains with the human author.

Scope and Non-Claims

This paper should be read as a conceptual and operational framework. It does not claim to provide a complete implementation, a formal proof of correctness, a universal verifier, or a guarantee that any AI agent will always make correct decisions.

The term *future self* is used operationally. It does not imply consciousness, personhood, or continuous subjective identity in AI systems. In this paper, a future self may refer to a later session, model, toolchain, memory state, or capability level that receives and continues a prior update.

Completion Integrity is not proposed as a replacement for CI/CD, software testing, deployment pipelines, audit systems, formal verification, or human decision ownership. CI/CD protects the artifact pipeline. Completion Integrity protects the self-recursive pipeline.

Companion Artifact

A companion software artifact is provided as a GitHub repository containing a JSON schema for the Minimal Completion Record, PASS/DELAY/BLOCK examples, a human-readable checklist, and a lightweight validator. This artifact is not a full implementation of self-evolving AI. Its purpose is only to make the Completion Gate inspectable and reusable as an operational template. The claims of this paper do not depend on the artifact.

Evidence and Versioning

Where deployed or cited operationally, Completion Integrity records should preserve evidence anchors such as source files, version identifiers, logs, hashes, timestamps, and As-of conditions. The paper recommends forward-only revision practice: substantive changes should be recorded as future deltas or addenda rather than silently rewriting prior releases.

Final Scope Statement

Decision-OS V12 defines one narrow claim:

> Completion is not proven correctness. Completion is future-restartable closure.

All claims in this paper should be interpreted within that scope.

References

[1] M. Nygard, "Documenting architecture decisions," Cognitect Blog, 2011, architecture Decision Records.

[2] National Institute of Standards and Technology, "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," U.S. Department of Commerce, National Institute of Standards and Technology, Tech. Rep. NIST AI 100-1, Jan. 2023. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf

[3] Google, "Secure ai framework," Google Safety and Security documentation, 2024, aI security and risk-management framework.

[4] Anthropic, "Responsible scaling policy," Anthropic policy document, 2025, aI risk governance and deployment safety framework.

[5] S. Nagata, "Decision-OS V5 Revised: SiriusA — A Zero-Knowledge Confirmation Layer for Trajectory-Aware Human–AI Decision Safety," 2026, preprint. Genesis DOI: 10.5281/zenodo.17480645.

[6] ——, "Decision-OS V6: Canonical Commitment Architecture for Non-Destructive Integration," 2026, preprint.

[7] ——, "Decision-OS V7: Aspire Intelligence for AGI — Operational Hook: Per-Update Gate with Single-Window Evaluation," 2026, preprint.

[8] ——, "Decision-OS V8: Time-Tube Control for Self-Safe AGI — Trajectory Control under Irreducible Mismatch," 2026, preprint.

[9] ——, "Decision-os v9.1: Impact-weighted release: From continuous-pass gates to condition-bound judgment reuse," 2026, preprint.

[10] ——, "Decision-os v10 note: Survival-bounded planning: Why optimization conflicts with survival," 2026, research note / preprint.

[11] ——, "Decision-os v11 note: Forget for evolution: Reconnectable forgetting for agentic ai," 2026, research note / preprint.

[12] ——, "Decision Constitution v1.1," 2026, operational constitution for temporal fidelity, As-of responsibility fixation, and forward-only change.